

GUIÓN DE EXPOSICIÓN}

No es un guion para seguir al pie de la letra, es para brindar amplia información de lo que puede ser tratado en la exposición.

Integrante 1 (Marcillo)

Buenas Tardes, con todos.

Hoy les vamos a presentar nuestro trabajo sobre Model-Driven Development, conocido como MDD, utilizando la herramienta BOUML. Para demostrar este enfoque tomamos como caso de estudio el módulo de Gestión de Citas Médicas del Sistema de Gestión Clínica Veterinaria con Inteligencia Artificial.

Antes de explicar la herramienta y mostrar el modelo que desarrollamos, es importante comprender la teoría que da origen a este tipo de desarrollo de software.

En el desarrollo tradicional, los programadores escriben gran parte del código desde cero y luego documentan el sistema. Sin embargo, este proceso puede generar errores, inconsistencias y hacer más difícil el mantenimiento del software cuando el proyecto crece.

Para resolver este problema surge la Ingeniería Dirigida por Modelos, conocida como MDE, que propone una idea diferente: en lugar de que el código sea el elemento principal del desarrollo, los modelos pasan a ser el centro de todo el proceso. Es decir, primero se diseña el sistema utilizando modelos bien estructurados y, posteriormente, esos modelos sirven como base para generar parte del código automáticamente. Esto permite mejorar la productividad, disminuir errores y mantener una documentación más organizada durante todo el ciclo de vida del software.

A partir de este paradigma aparece la Arquitectura Dirigida por Modelos, o MDA, propuesta por la Object Management Group. Su objetivo es separar la lógica del negocio de la tecnología con la que finalmente se implementará el sistema. De esta manera, un mismo diseño puede adaptarse a diferentes plataformas sin necesidad de volver a construirlo desde cero.

Para lograrlo, MDA organiza el desarrollo en tres niveles de abstracción.

El primero es el CIM, que representa los requisitos del negocio y describe qué necesita el usuario sin pensar todavía en aspectos técnicos.

El segundo es el PIM, donde ya se modela el funcionamiento y la estructura del sistema, pero aún sin depender de un lenguaje de programación o de una plataforma específica.

Finalmente encontramos el PSM, que adapta ese modelo a una tecnología concreta, como puede ser C++, Java o Python. Gracias a esta separación es posible reutilizar el diseño y facilitar el mantenimiento del software.

Todo esto nos lleva al concepto principal de nuestro trabajo: el Model-Driven Development, o MDD.

MDD es la aplicación práctica de estos principios. Su funcionamiento consiste en construir modelos UML y transformarlos automáticamente en código mediante herramientas especializadas. En lugar de programar toda la estructura manualmente, el desarrollador crea primero un modelo del sistema y luego una herramienta se encarga de generar gran parte del código inicial. Esto no reemplaza completamente al programador, ya que todavía es necesario implementar la lógica del negocio, pero sí agiliza considerablemente el desarrollo y mantiene una mayor coherencia entre el diseño y la implementación.

En nuestro proyecto aplicamos precisamente esta metodología para modelar un sistema veterinario y generar automáticamente su estructura en C++ utilizando la herramienta BOUML.

Ahora que conocemos los fundamentos del desarrollo dirigido por modelos, mi compañera explicará el lenguaje de modelado UML, la herramienta BOUML y cómo aplicamos estos conceptos en nuestro proyecto.

Integrante 2 (Robinson)

Muchas gracias, compañero.

Como acabamos de escuchar, el desarrollo dirigido por modelos se basa en construir primero un modelo del sistema antes de generar el código. Sin embargo, para que esto sea posible es necesario utilizar un lenguaje que permita representar gráficamente ese sistema. Ese lenguaje es UML, que significa *Unified Modeling Language* o Lenguaje Unificado de Modelado.

UML es un estándar utilizado en la ingeniería de software para representar de forma visual la estructura y el comportamiento de un sistema. Su principal ventaja es que facilita la comunicación entre los integrantes del equipo, ya que todos pueden comprender cómo está diseñado el software antes de comenzar a programarlo.

Dentro del enfoque MDD, UML tiene un papel muy importante porque es el lenguaje que se utiliza para construir los modelos que posteriormente pueden transformarse en código.

Además del lenguaje UML, durante el desarrollo también es necesario realizar un adecuado modelado de requisitos. Esto consiste en representar el sistema desde diferentes perspectivas para comprender completamente su funcionamiento.

La primera perspectiva es el modelo de datos, que responde a la pregunta: *¿qué elementos existen dentro del sistema?* En nuestro caso encontramos entidades como usuarios, veterinarios, clientes, mascotas, historiales clínicos y atenciones médicas.

La segunda perspectiva corresponde al modelo funcional, que explica cómo interactúan esos elementos y qué procesos realiza el sistema, por ejemplo registrar una atención clínica, consultar una cita o generar una sugerencia mediante inteligencia artificial.

Finalmente está el modelo de comportamiento, que describe el orden en que ocurren las acciones y cómo cambian los estados del sistema durante su funcionamiento.

Una vez definidos estos modelos utilizamos la herramienta BOUML para construir el proyecto.

BOUML es una herramienta de modelado UML de código abierto que permite diseñar sistemas orientados a objetos y, además, generar código automáticamente a partir de esos modelos. Fue desarrollada como una alternativa ligera y rápida frente a otras herramientas de modelado, por lo que resulta muy útil tanto en proyectos académicos como profesionales.

Entre sus principales características podemos mencionar que es gratuita, funciona en Windows, Linux y macOS, permite organizar grandes proyectos mediante paquetes y soporta la generación de código en lenguajes como C++, Java, Python y PHP. Además, también ofrece funciones de ingeniería inversa y generación de documentación técnica, lo que facilita mantener actualizado el proyecto.

Para demostrar el funcionamiento de esta herramienta utilizamos como caso de estudio nuestro Sistema de Gestión Clínica Veterinaria con Inteligencia Artificial, específicamente el módulo de Gestión de Citas y Atenciones Clínicas. Elegimos este módulo porque reúne varias entidades relacionadas entre sí y permite aplicar correctamente los principios del desarrollo dirigido por modelos.

En este módulo modelamos diferentes clases, entre ellas Usuario, Veterinario, Cliente, Mascota, Atención Clínica, Historial Clínico, Sugerencia IA, Inventario y Notificación. También elaboramos un diagrama de casos de uso, que representa las funciones que realizan los actores del sistema, y un diagrama de estados, que muestra cómo cambia el estado de una atención clínica durante su ciclo de vida. Estos diagramas nos permitieron representar tanto la estructura como el comportamiento del sistema antes de generar el código.

Una vez construido todo el modelo UML, el siguiente paso fue analizar por qué elegimos BOUML frente a otras herramientas y cómo aprovechamos sus características para generar automáticamente el código del sistema. Sobre ese tema continuará mi compañero.

Integrante 3 (Arteaga)

Muchas gracias, compañero.

Una vez que definimos el modelo UML de nuestro sistema, el siguiente paso fue seleccionar la herramienta con la que íbamos a trabajar. Existen varias herramientas de modelado, pero después de analizar diferentes opciones decidimos utilizar BOUML, ya que era la que mejor se adaptaba a los objetivos de esta práctica.

Una de las principales ventajas de BOUML es que se trata de un software de código abierto y completamente gratuito, lo que facilita su utilización tanto en instituciones educativas como en proyectos personales. Además, es una herramienta multiplataforma, por lo que puede ejecutarse en sistemas Windows, Linux y macOS sin necesidad de realizar modificaciones importantes.

Otra característica importante es que consume pocos recursos del computador, por lo que funciona de forma rápida incluso en equipos con especificaciones básicas. También permite organizar proyectos mediante paquetes, generar documentación y soporta varios lenguajes de programación, entre ellos C++, Java, Python, PHP e IDL. Esto le da una gran flexibilidad para trabajar en diferentes entornos de desarrollo.

Sin embargo, como toda herramienta, BOUML también presenta algunas limitaciones. Su interfaz gráfica es bastante sencilla y menos moderna que la de otros programas de modelado. Además, la documentación oficial y la comunidad de usuarios son más reducidas, lo que puede dificultar encontrar información o resolver problemas específicos. Finalmente, recibe pocas actualizaciones en comparación con herramientas más comerciales.

A pesar de estas desventajas, para nuestro trabajo resultó ser una excelente opción, ya que el objetivo principal era aplicar el enfoque Model-Driven Development y comprender el proceso de generación de código a partir de modelos UML, más que utilizar una herramienta con una interfaz moderna.

También realizamos una comparación con otras herramientas conocidas como StarUML y Visual Paradigm. StarUML ofrece una interfaz más atractiva y soporte para distintos lenguajes, mientras que Visual Paradigm incorpora funciones avanzadas de colaboración y modelado empresarial. Sin embargo, muchas de esas características requieren licencias de pago o versiones comerciales. En cambio, BOUML nos permitió realizar todo el proceso de modelado y generación de código de manera gratuita, con un excelente rendimiento y sin restricciones para el desarrollo académico.

Otro aspecto importante es que BOUML permite realizar ingeniería directa, también conocida como *Forward Engineering*. Esto significa que, una vez terminado el modelo UML, la herramienta puede utilizar ese diseño para generar automáticamente la estructura inicial del código fuente. En nuestro caso seleccionamos C++ como lenguaje objetivo, ya que es uno de los lenguajes soportados de forma nativa por BOUML y era el más adecuado para cumplir con los requisitos de la práctica.

Es importante aclarar que la generación automática no crea un programa completamente terminado. Lo que hace es construir la estructura de las clases, los atributos, los métodos y las relaciones definidas en el modelo UML. Posteriormente, el desarrollador debe implementar la lógica de negocio correspondiente, es decir, el comportamiento específico que tendrá cada función del sistema.

Gracias a este proceso logramos mantener una correspondencia entre el diseño realizado en UML y el código generado, facilitando la organización y el mantenimiento del proyecto.

Finalmente, mi compañero explicará cómo se llevó a cabo esta generación de código, el concepto de Roundtrip Engineering y las principales conclusiones obtenidas durante el desarrollo de esta práctica.

Integrante 4 (Herrera)

Muchas gracias, compañera.

Como acabamos de escuchar, una de las principales funciones de BOUML es la generación automática de código a partir de modelos UML. Sin embargo, esta no es la única característica importante de la herramienta. También incorpora otras funcionalidades que ayudan a mantener sincronizados el diseño y la implementación del software durante todo el proceso de desarrollo.

Una de ellas es la ingeniería inversa, o *Reverse Engineering*. A diferencia de la ingeniería directa, donde el modelo se transforma en código, la ingeniería inversa realiza el proceso contrario: analiza un código existente para reconstruir el modelo UML correspondiente. Esto resulta muy útil cuando se trabaja con proyectos que ya fueron desarrollados y no cuentan con documentación actualizada, ya que permite recuperar la estructura del sistema y comprender más fácilmente su funcionamiento.

Otro concepto muy importante es el Roundtrip Engineering, también conocido como ingeniería bidireccional. Este mecanismo combina la ingeniería directa y la ingeniería inversa para mantener sincronizados el modelo UML y el código fuente. En otras palabras, los cambios realizados en el modelo pueden reflejarse en el código y, de igual manera, algunas modificaciones realizadas en el código pueden actualizar el modelo. Esto ayuda a evitar inconsistencias y facilita el mantenimiento del software cuando el proyecto evoluciona con el tiempo.

Durante nuestra práctica pudimos comprobar este funcionamiento. Después de generar el código inicial, realizamos modificaciones en el modelo UML, como la incorporación de un nuevo atributo dentro de una de las clases. Al volver a ejecutar la generación de código, BOUML actualizó automáticamente la estructura correspondiente sin afectar la organización general del proyecto. Con esta prueba verificamos que la herramienta mantiene la relación entre el diseño y la implementación, que es precisamente uno de los principios fundamentales del desarrollo dirigido por modelos.

A partir de esta experiencia también identificamos algunos beneficios del enfoque Model-Driven Development. En primer lugar, permite reducir el tiempo necesario para desarrollar la estructura inicial de un sistema. Además, mejora la calidad del diseño, mantiene una documentación más organizada y facilita el mantenimiento del software, ya que cualquier cambio importante comienza en el modelo y luego puede reflejarse en el código. Estas características hacen que el proceso de desarrollo sea más ordenado y disminuyen la posibilidad de cometer errores cuando el proyecto aumenta de tamaño.

No obstante, también encontramos algunas limitaciones. Aunque herramientas como BOUML automatizan gran parte del trabajo, todavía es responsabilidad del desarrollador implementar la lógica de negocio, validar la información y adaptar el código a las necesidades específicas del sistema. Además, el uso de MDD requiere conocer correctamente UML y comprender cómo construir modelos de calidad antes de generar el código.

Como conclusión, podemos afirmar que esta práctica nos permitió comprobar que el desarrollo dirigido por modelos constituye una metodología muy útil para organizar proyectos de software. Gracias al uso de UML y de BOUML fue posible diseñar el sistema antes de programarlo, generar automáticamente una parte importante de su estructura y mantener una relación consistente entre el modelo y el código.

Finalmente, consideramos que este tipo de herramientas no busca reemplazar al desarrollador, sino brindarle apoyo para trabajar de una forma más eficiente, organizada y con una mejor documentación del proyecto.